

Project Proposal — «GigaSpec»: самоеволюционирующий SDD-оркестратор на Ouroboros

Инициатива: «GigaSpec» — оркестратор SDD-конвейера команды (сигнал → готовое ТЗ → код → PR) поверх Ouroboros и MCP Atlassian, с самоеволюцией процесса. **Команда:** Гуси-Лебеди · **Дата:** 4 июля 2026

1. Описание кейса

Зачем (проблема)

В компании SDD (Спец-Driven Development) нужен — качество, предсказуемость и аудируемость требуют описанного процесса. Но команды внедрять его не рвутся: для них это регламенты и бюрократия, а описать, **как процесс выглядит** именно у них, не может никто. Следствия:

- регламенты на 40 страниц никто не читает;
- внедрение процесса буксует на этапе «давайте сначала договоримся, как мы работаем»;
- сырые задачи уходят в разработку → возвраты, переделки, бесконечные «а что ты имел в виду?»;
- готовые код-агенты стартуют на мутных задачах и выдают правдоподобный, но неверный код, который дольше проверять, чем писать заново;
- знание процесса живёт в головах сеньоров и теряется при онбординге.

Что (суть инициативы)

Самоеволюционирующий SDD-оркестратор: ловит сигнал из любого канала (задача в Jira, тег агента в чате, встреча в **Jazz**), проводит его через SDD-процесс *вашей* команды по этапам — **готовность (ТЗ) → аналитика → дизайн → код → PR** — с гейтами и human-in-the-loop на каждом стыке. Мы ведём задачу через **весь цикл — от сигнала до PR**; каждый этап при этом самостоятельно ценен, проходит через апрув человека и чек-поинт (можно вернуться и продолжить).

Ключевая идея: **процесс не описываем — добываем из команды: интервью + её живые источники**. Порог входа стремится к нулю.

Формула продукта — пять глаголов, замкнутый цикл:

Добывает → Применяет → Охраняет → Учит → Эволюционирует.

Один источник правды — редактируемые `.md`-файлы (чек-лист готовности + шаблоны + описание этапов). Один движок работает для всех ролей — меняется только чек-лист «что значит готово»:

Роль	Вход	Выход (готово, когда...)
Аналитик	сырая гипотеза	T3 передаётся в разработку без вопросов
Продакт	идея фичи	PRD согласован, метрики и score зафиксированы
Дизайнер	задача на макет	бриф + критерии для проектирования
Разработчик	готовое T3	PR на зелёном quality gate

Как

- 1. Калибровка** — интервью с командой (скилл спрашивает, как опытный тимлид на онбординге)
 - о скан всех доступных источников (бэклог Jira, Confluence, чаты) → неявный стандарт команды → чек-лист, шаблоны и этапы конвейера в `.md`.
- 2. Конвейер** — сигнал → задача → петля готовности (черновик → самокритика по чек-листу → уточняющие вопросы человеку) → готовое T3 → следующие этапы по процессу команды, вплоть до кода в изолированной ветке и PR на зелёном quality gate.
- 3. Выгрузка** — артефакты этапов в Confluence/Jira, метка **Ready**, PR в репозитории.

Отстройка

- **Ров:** стандарт добыт из **вашей** команды (интервью + сверка со всеми источниками) + замкнутая петля самокритики (не разовая генерация) + эволюция от спринта к спринту.
- **Не код-агент, а управляемый конвейер:** код не стартует на мутной спеке (интервью-гейт), «готово» агента проверяется независимым quality gate, у каждого действия есть апрув и откат.
- **База — это мнение:** жёсткий формат, мягкое содержание. Команда спорит с конкретным чек-листом → в споре находит свой процесс. «Гибкая база» = пустота, мы даём позицию.
- **Самозволюция процесса, а не модели:** эволюционирует описание процесса (`.md`) через `diff` и апрув человека — для банка это не риск, а усиление контроля.

2. Эффект от внедрения

Продаём не скорость, а качество и предсказуемость: меньше переделок, единый процесс, аудируемость. Эти эффекты **не зависят от скорости инференса** — сохраняются и на моделях внутреннего контура (GigaChat / Cloud.ru). Прогнозных рублей не рисуем — вместо этого даём методику пересчёта эффекта в деньги на фактических данных команды (ниже).

За счёт чего достигается эффект

- 1. Меньше переделок спеки** — дефект ловится до разработки, а не в готовом PR. Правило **1:10:100**: дефект на спеке на порядок дешевле дефекта в коде и на два

порядка дешевле прод-инцидента.

2. **Меньше переделок кода** — этап кода не стартует на мутной задаче (clarify-гейт); PR открывается только на зелёном quality gate (обязательные тесты/линт/сборка).
3. **Меньше времени на груминг** — рутину уточнений берёт скилл, сеньор подключается точно.
4. **Быстрее онбординг** — новичок спрашивает «как у нас принято?» и получает ответ на примерах.
5. **Ноль стоимости внедрения процесса** — не пишем регламент: одно интервью + автоскан источников → готовый стандарт.
6. **Качество растёт со временем** — самоэволюция под ретро-гейт режет пул повторяющихся переделок от спринта к спринту (отложенный множитель эффекта).

Что меняется (качественно)

Что	Сейчас	С GigaSpec
Готовность задачи к работе	мутные задачи уходят в разработку, уточнения теряются в чатах и устно	задача не стартует, пока критичные вопросы не закрыты — всё зафиксировано в самой задаче
Возвраты и переделки	дефект всплывает уже в готовом PR	дефект ловится на спеке, до кода (правило 1:10:100)
Единство процесса	у каждого «свой» негласный процесс	один описанный стандарт, единый для всех задач
Онбординг	знание живёт в головах сеньоров	новичок спрашивает «как у нас принято?» и получает ответ на примерах команды
Внедрение процесса	месяцы согласований + регламент на 40 страниц	~0: интервью + автоскан источников → стандарт с первого дня

Прогнозный ROI намеренно не приводим: продаём качество и предсказуемость, которые не зависят от скорости инференса и сохраняются на моделях внутреннего контура. Деньги при этом считаемы — методика ниже заполняется фактами пилота, а не оценками «на глаз».

Как перевести в деньги

Экономия за спринт $\approx N$ задач $\times$ доля возвратов $\times$ ср. часы переделки $\times$ ставка часа – стоимость токенов.

Все множители у команды уже есть в Jira (число задач, реопены, время в статусах); стоимость токенов Ouroboros считает из коробки (Dashboard → Costs). После 1–2 спринтов пилота формула заполняется фактическими данными команды, а не прогнозом — поэтому готовых рублей здесь нет, есть способ их честно посчитать.

Стоимость под контролем

Готовый ROI-расчёт не приводим (методика — выше), но стоимость работы агента прозрачна и управляема: Ouroboros ведёт учёт цены токенов из коробки (Dashboard → Costs) — фактическая цена каждого прогона видна с первого дня, а на задачу/этап можно поставить **бюджет-лимит**. Детерминированные проверки (наличие полей/меток/секций) сделаны кодом, не LLM — это держит расход в рамках лимита и не зависит от цен на инференс.

Нефинансовые эффекты

- **AI-adoption — стратегический KPI Банка:** каждая задача через конвейер — управляемое и аудируемое применение AI в разработке (растёт доля AI-разработки под контролем, а не «теневое» использование ассистентов).
- **Аудируемость под банковскую регуляторику:** стоп-гейты **Always / Ask / Never** + логи всех действий + апрув человека — у каждого изменения есть diff, причина, апрувер и откат.
- **Предсказуемость поставки:** единый процесс для всех задач, меньше дефектов доезжает до прод (обязательные проверки в quality gate).

Тиражируемость эффекта

Эффект мультиплицируется: один скилл → любое число команд без доработки.

3. Процесс работы агента

Пошаговый сценарий

Шаг 0. Калибровка через интервью (один раз на команду). Скилл не с нуля — в него заложена база «каким должен быть хороший процесс» (обязательные атрибуты задачи, этапы конвейера). Главный инструмент — **структурированное интервью**: скилл расспрашивает команду, как опытный тимлид на онбординге, — продукт, роли, этапы, что значит «готово», где болит. Параллельно **сканирует все доступные источники** (бэклог Jira целиком, Confluence, чаты) — не как эталон (будем честны: задачи в трекере обычно описаны слабо — это и есть боль, которую решаем), а как свидетельства практики: находит расхождения между «как говорим» и «как делаем» и превращает их в вопросы интервью. Итог → чек-лист, шаблоны и этапы в `checklist.md` / `template.md`. Команда правит `.md` — это вся настройка, без кода. **Команде без истории** (новый продукт, новый состав) хватает базы + интервью: стандарт есть с первого дня, петля эволюции дозатачивает его по мере накопления собственных задач.

Шаг 1. Захват сигнала. Скилл заводит задачу сам — из чата или из встречи в **Jazz** (по тегу агента вытягивает суть в готовую задачу по процессу) — либо тянет существующую задачу из Jira через MCP Atlassian + контекст: связанные задачи, эпик, похожие прошлые. Проверяет на дубли.

Шаг 2. Этап готовности (первый гейт конвейера).

draft T3 по template.md

- самокритика по checklist.md → чего не хватает
- уточняющие вопросы человеку (human-in-the-loop)
- переписать с учётом ответов → готовое T3

Пока критичные вопросы спеки не закрыты человеком — следующие этапы не стартуют.

Шаг 3. Выгрузка и следующие этапы. Готовое T3/PRD → Confluence; дозаполнение полей Jira; метка **Ready**. Дальше задача идёт по этапам процесса команды: аналитика → дизайн → **код** (изолированная ветка/worktree, независимый quality gate: тесты + линт + сборка) → **PR открывается только на зелёной проверке**. Каждый этап — точка апрува человека (дальше / поправить / стоп), но целимся мы в сквозной проход до PR.

Шаг 4. Охрана и эволюция (фоновое, по расписанию). Мониторит очередь Jira: в заведённой задаче сверяет с процессом и, если чего-то не хватает, **пишет комментарий в самой задаче и пингует автора** в подключённом канале (для демо — Telegram). Копит сигналы (реопены, доправки) → раз в спринт смотрит на задачи команды (что влетало, как менялось, реопены) и предлагает `diff` к `.md` → тимлид апрувит.

Как встраивается в текущий процесс

- Поверх **существующих** Jira, Confluence и репозитория — новые инструменты команде не нужны.
- **Каналы — агностичны:** сигнал на вход из любого подключённого канала (тег в чате, встреча, Jira, командная строка), обратная связь — в любой канал, куда Ougoboros умеет писать (MCP/транспорты). Дефолт для аудируемости — **комментарий в Jira-задаче** (история в задаче)
 - пинг автора в чате со ссылкой. В прототипе показываем Jira + Telegram.
- **Human-in-the-loop** на каждом стыке этапов: «дальше / поправить / стоп». Не чёрный ящик.
- Скилл **никогда не меняет процесс сам** — только предлагает, апрув человека обязателен.
- Инженерные гарантии доверия: явное состояние вне сессии LLM (resume), изоляция задач в ветке/worktree, независимый quality gate, лимиты на попытки/токены, видимая стоимость, курированная память.

4. Функции прототипа / MVP

Прототип — **конвейер с явными этапами**, коммит на полный цикл **идея → PR**. Цель прототипа — сквозной проход сигнал → готовность → выгрузка → код → PR плюс охрана и самоэволюция. Каждый этап проходит через апрув человека и чек-

поинт (стейт-машина: можно вернуться к любому шагу и продолжить). Этап кода на демо — отрепетированный happy-path прогон до PR; устойчивость на произвольных задачах — за рамками MVP (обозначено честно).

Ядро MVP (обязательный скоуп)

- 0. **Самозволюция** — ретро процесса команды по её SDD: `signals.log` (≥2 источника) → дайджест → `diff` к `.md` → апрув человека.
- 1. **Калибровка-интервью** — база «хорошего процесса» + интервью с командой + скан источников → `.md`.
- 2. **Работа с MCP Atlassian + Telegram** — чтение/запись Jira и Confluence (метка `Ready`), сигналы и пинги в канал.
- 3. **Полный процесс идея → PR (SDD)** — готовность → аналитика → дизайн → код → PR на зелёном quality gate; этап готовности = черновик → самокритика по чек-листу → вопросы человеку.
- 4. **Чек-поинты (стейт-машина)** — состояние каждого этапа сохраняется вне сессии LLM; можно вернуться к любому шагу и продолжить (устойчивость к обрыву модели).
- 5. **Дашборд + мониторинг** — вкладка в web-UI Ouroboros: мониторинг выполненных задач, логи с метриками (здоровье pipeline).
- 6. **Классификатор задачи** — баг / фича / ... → свой процесс под тип задачи.

Три главные фичи (звёзды прототипа)

- 1. **Заводит задачи сам, по вашему процессу** — тег агента в чате или встреча из **Jazz** → готовая задача в Jira по стандарту команды.
- 2. **Охраняет входящие** — мониторинг очереди Jira по расписанию → сверка с процессом → авто-коммент автору о том, чего не хватает.
- 3. **Самозволюция процесса** — раз в спринт анализ задач команды → предложение улучшить процесс (`diff` к `.md` → апрув человека).

Слой 1 — усилители (если успеем)

Фича	Ценность
Три вопроса сеньора	ровно 3 главных уточнения, не форма из 20 полей
«Это дубль!»	находит похожую/дублирующую задачу в бэклоге
DoR-линтер	вставь задачу → «готовность 40%, нет критериев приёмки»
Readiness score	наглядный рост готовности задачи
Доска процесса	kanban задач по этапам (<code>ui_tab</code> поверх дашборда ядра): где каждая задача, гейты, следующее действие
Стоимость задачи	видимая цена токенов per задача/этап + бюджет-лимит (учёт — штатный в Ouroboros) — управляемость затрат, не обещание экономии

Целевое видение (рассказываем, не обещаем в MVP)

UI-конструктор процесса из блоков (этапы/роли/гейты/границы Always·Ask·Never → конфиг Ouroboros) — прорабатывается отдельной веткой, в гарантированный скоуп MVP не входит. За рамками MVP также: story points по аналогии и устойчивый этап кода на произвольных задачах (за пределами happy-path).

5. Ограничения

Источники данных

- **Jira** — задачи, поля, статусы, комментарии, связи (через MCP Atlassian).
- **Confluence** — выгрузка артефактов, метки.
- **Репозиторий (GitLab/аналог)** — ветки, PR, результаты проверок (этап кода).
- **Каналы связи** (Telegram / Slack / СберЧат / ...) — вход сигналов и обратная связь; каналы агностичны, подключается любой доступный Ouroboros транспорт. В прототипе — Telegram.
- Данные — **внутренний контур команды**; скилл не требует внешних датасетов.

Кибербезопасность

- **Ouroboros — внутренняя платформа Банка** и нативно поддерживает модели внутреннего/российского контура (GigaChat, Cloud.ru Foundation Models) → в проме чувствительные данные могут не покидать контур. Внешние LLM (OpenRouter) — только на хакатоне, с контролем передаваемых данных и маскированием/исключением чувствительных полей.
- Изоляция задач в песочнице (ветка/worktree), чекпойнты и откат.
- **Границы автономии Always / Ask / Never:** чтение — свободно; запись / ехес / push — через Approval Gate; разрушительные операции — заблокированы на уровне платформы (промт-инъекция границу не обходит).
- **Апрув человека** на все изменения → нет автономных действий с необратимыми последствиями; PR — только через независимый quality gate.
- Доступы через сервисную учётку с минимально необходимыми правами (least privilege); MCP-инструменты в Ouroboros проходят per-call safety-check, токены маскируются.

Регуляторные ограничения

- Банковская тайна и ПДн в задачах → обработка в рамках внутренних политик Банка; при необходимости — обезличивание (в т.ч. расшифровок встреч перед LLM).
 - Аудируемость: обратная связь — комментарии в Jira (история в задаче), все предложения скилла логируются и проходят ревью человека → трассируемость решений.
-

6. Реализуемость

Пошаговый процесс реализации прототипа на Ouroboros

Всё перечисленное — **штатные механизмы платформы Ouroboros**, наш код — конфигурация, промпты и тонкий plugin-слой:

1. **Скилл** — тип `extension` (манифест `SKILL.md` + `plugin.py` через PluginAPI); процесс команды — в `.md`-файлах скилла.
2. **MCP Atlassian** — через встроенный MCP-клиент Ouroboros (настройка `MCP_SERVERS`, транспорт streamable HTTP/SSE): чтение задач, поиск, комментарии, выгрузка в Confluence.
3. **Фоновый мониторинг очереди** (фича «охраняет») — блок `scheduled_tasks` (cron) в манифесте скилла; планировщик Ouroboros сам будит агента по расписанию. Тот же механизм — для спринтового дайджеста эволюции.
4. **Канал связи для демо** — telegram-bridge, штатный transport-скилл Ouroboros (тег агента в чате как сигнал, ответы/пинги туда же); архитектура канал-агностична — подключается любой транспорт/MCP.
5. **Роли-агенты конвейера** — субагенты Ouroboros: model lanes (main/heavy/light/review) под вес этапа, изоляция каждого acting-субагента в **git-worktree** и интеграция патча с проверкой — из коробки.
6. **Петля готовности + калибровка** — промпты ролей + `checklist.md` / `template.md`.
7. **Наблюдаемость и стоимость** — встроенный Dashboard Ouroboros (Logs / Costs / Activity), учёт цены токенов на задачу.
8. Подготовка демо-данных (сид) и записанный фолбэк.

Масштабируемость (в рамках подразделения)

Единый движок, вся специфика — в `.md`. Новая команда = своё интервью + свои источники → свой стандарт. Работает и для команды **без истории** (новый продукт, свежий состав): стартует с базы «хорошего процесса» + интервью, эволюция дозатачивает стандарт по мере накопления задач. Никакого кода под каждую команду.

Переиспользование (другими подразделениями)

Механика «интервью + скан ваших источников → ваш процесс» работает для любого подразделения (аналитика, продакт, дизайн, разработка) без изменения кода — меняется только чек-лист и набор этапов.

Риски и нивелирование

Риск	Нивелирование
Источники слабые (задачи в треkere описаны плохо) или истории нет вовсе (новая команда)	Главный источник — интервью, скан лишь сверяет слова с практикой; база «хорошего процесса» как старт; тимлид правит <code>.md</code> ; петля эволюции дозатачивает
Этап кода нестабилен на произвольных задачах	В демо — отрепетированный happy-path прогон до PR; устойчивость на любых задачах за рамками MVP (обозначено честно)
Чувствительные данные в задачах	Внутренний контур (GigaChat/Cloud.ru в проме), обезличивание, апрув человека, least privilege
Недоверие команды к агенту	Прозрачный diff-коммент, видимая стоимость, «стоп» на каждом шаге
Галлюцинации / неверные уточнения	Порог readiness score, лимит итераций, независимый quality gate, финальный апрув
Сбой на демо	Сид данных заранее + записанный фолбэк